9장 복귀

깃은 소스 코드의 시간에 따른 변화 흐름을 기록합니다. 이렇게 저장된 변화 흐름 단계를 커밋(commit)이라고 합니다. 이 장에서는 커밋을 기준으로 특정 시점의 상태로 되돌리는 방법을 알아보겠습니다.

9.1 되돌리기

9.2 리셋

9.3 리버트

9.4 정리

9.1 되돌리기

깃을 이용하여 버전을 관리하는 목적은 만일의 사태를 대비하기 위해서입니다. 프로그래밍은 수많은 코드를 작성하고 테스트하는 과정입니다. 개발자는 코드를 단계별로 발전시키면서 실수를 최소화하고자 노력합니다. 하지만 아무리 주의해서 프로그래밍해도 오류가 생깁니다. 개발 도중에 심각할 정도로 오작동해서 더 이상 개선하기 어려울 때도 생기곤 합니다. 이때는 문제를 억지로 해결하려고 노력하는 것보다 지금까지 한 작업을 포기하고 다시 시작하는 것이 좀 더 빠르게 해결할 수 있는 방법이 될 수도 있습니다.

깃을 사용하면 언제든지 원하는 시점으로 전체 코드를 되돌릴 수 있습니다. 마치 코드 복구 시스템과 같습니다.

9.1.1 다시 시작

코드가 한두 줄 변경되었을 때는 언제든지 다시 시작할 수 있습니다. 몇 줄만 암기하여 변경하면 됩니다. 하지만 몇 시간 혹은 며칠 동안 작업한 코드라면 이야기가 다릅니다. 모든 수정 내역을 일일이 기억하여 과거 상태로 되돌아가는 것은 쉽지 않습니다. 이때 깃을 사용하면 좀 더 손쉽게 되돌아갈 수 있습니다.

깃은 기록된 커밋을 기준으로 이전 상태로 되돌릴 수 있습니다. 즉, 코드를 항상 안정적으로 유지하고 관리할 수 있게 해 주는 도구입니다. 수많은 개발자가 깃을 사랑하는 이유 중 하나이기도 합니다.

깃에서 코드 작업을 되돌리는 방법은 크게 reset과 revert 두 가지입니다. 리셋(reset)과 리버트(revert) 동작을 좀 더 쉽게 이해할 수 있도록 실습으로 익혀 보겠습니다. 먼저 실습을 위해 새 깃 저장소 폴더를 만들고 초기화합니다.

```
$ cd 실습폴더


$ mkdir gitstudy09 새 폴더 만들기


$ cd gitstudy09




infoh@DESKTOP MINGW64 /e/gitstudy09

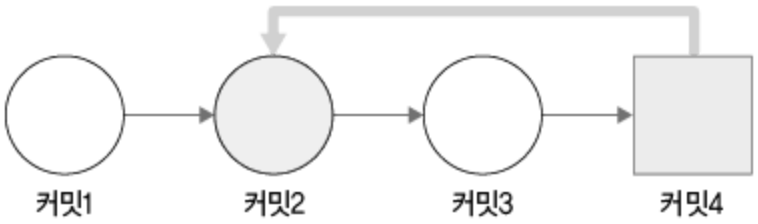

$ git init  저장소 초기화


Initialized empty Git repository in E:/gitstudy09/.git/
```

9.2 리셋

리셋(reset)은 커밋을 기준으로 이전 코드로 되돌리는 방법으로, 기록한 커밋을 취소합니다. 커밋을 취소하는 만큼 리셋할 때는 항상 신중하게 작업해야 합니다.

▼ 그림 9-2 리셋



9.2.1 복귀 시점

복귀는 어떤 시점으로 돌아가는 것을 의미합니다. 따라서 이전 코드로 복귀하려면 복귀 시점을 알려 주어야 합니다. 리셋은 이 시점을 커밋을 기준으로 정합니다. 커밋은 log 명령어로 조회할 수 있습니다. log 명령어를 실행하면 커밋의 해시 값과 메시지를 출력합니다. 따라서 복귀하고자 하는 특정 시점을 찾는 데 매우 유용합니다.

앞에서 작업한 커밋들을 확인해 봅시다. --oneline 옵션을 같이 사용하여 로그 기록을 확인합니다. 간략한 커밋 해시 값과 메시지가 같이 출력됩니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy09 (main)


$ git log --oneline 로그 확인



7f068b6 (HEAD → main) menu5


6619c99 menu4


b728366 menu3


f1c704f menu2


b741eef menu1


69bf712 first
```

9.2.2 reset 명령어

reset 명령어를 사용하면 지정된 커밋 코드로 되돌아갑니다. 즉, 특정 커밋의 해시 값 상태로 모든 코드를 복구합니다.

$ git reset 옵션 커밋ID

reset 명령어는 옵션을 함께 사용해야 하며, 세 가지 옵션이 있습니다.

- soft: 스테이지 영역을 포함한 상태로 복원합니다.
- mixed: 기본 옵션 값은 mixed입니다. reset 명령어를 사용할 때 옵션을 지정하지 않으면 기본값인 mixed로 선택됩니다.
- hard: 실제 파일이 삭제된 이전 상태로 복원합니다.

soft 옵션과 mixed 옵션 차이는 크게 스테이지 영역과 관련이 있습니다. hard 옵션은 워킹 디렉터리와 관련이 있습니다. 각 옵션의 차이점과 동작은 실습하면서 알아보겠습니다.

9.2.3 soft 옵션

soft 옵션은 가장 낮은 단계의 리셋 동작입니다. 먼저 soft 옵션의 동작은 reset --soft 명령어를 실행해서 이해해 봅시다. 코드에서 HEAD~는 이전 커밋을 의미합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy09 (main)


$ git reset --soft HEAD~ 이전 커밋으로 soft 옵션을 사용한 리셋
```

최신 커밋(menu5)을 리셋했습니다. --soft 옵션을 사용하면 별도의 메시지가 출력되지 않습니다. 리셋한 후 소스트리에서 확인하면 다음 그래프를 볼 수 있습니다.

▼ 그림 9-4 소프트 리셋 확인



마지막 menu5 커밋이 사라지고 커밋하지 않은 변경 사항이 보입니다.

9.2.4 mixed 옵션

reset 명령어의 기본값은 mixed 옵션입니다.

$ git reset --mixed 커밋ID

또는 다음과 같이 mixed 옵션을 생략할 수 있습니다.

$ git reset 커밋ID 생략 가능

이번에는 mixed 옵션을 이해할 수 있도록 간단히 실습해 봅시다.

```
infoh@DESKTOP MINGW64 /e/gitstudy09 (main)


$ git reset --mixed HEAD~ mixed 옵션을 사용한 리셋 실행


Unstaged changes after reset:


M       menu.htm
```

이번에는 리셋 메시지가 같이 표시되었습니다. 메시지 내용은 unstaged 상태로 변경되었다는 의미입니다. mixed 옵션은 soft 옵션과 달리 리셋한 후 스테이지 상태까지 복원하지 않습니다. status 명령어로 깃의 상태를 확인합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy09 (main)

$ git status 상태 확인

On branch main

Changes not staged for commit:

  (use "git add <file>..." to update what will be committed)

  (use "git checkout -- <file>..." to discard changes in working directory)

  modified:   menu.htm

no changes added to commit (use "git add" and/or "git commit -a")
```

9.2.5 hard 옵션

이번에는 가장 강력한 옵션인 hard 옵션을 알아봅니다. soft 옵션과 mixed 옵션은 이전 커밋으로 되돌리는 과정에서 실제로 삭제되는 내용은 없습니다. 모든 작업 내용이 워킹 디렉터리에 그대로 유지됩니다. 정확히 말하면 이전 상태로 되돌아가는 것이 아닙니다. 복귀 커밋 이후의 작업들은 워킹 디렉터리 영역에 남겨 두어 다시 수정하거나 커밋할 수 있도록 상태만 변경하는 것입니다.

이에 반해 hard 옵션은 리셋되는 복귀 시점의 커밋 상태와 해당 커밋의 워킹 디렉터리까지 모두 되돌립니다. 즉, reset --hard 명령어를 사용한 커밋 이후의 모든 내용은 삭제됩니다. 따라서 hard 옵션은 주의해서 사용해야 합니다.

이번에는 hard 옵션을 사용해 봅시다.

```
infoh@DESKTOP MINGW64 /e/gitstudy09 (main)


$ git reset --hard HEAD~ 완전 삭제


HEAD is now at 6619c99 menu4
```

hard 옵션을 실행하면 리셋된 결과 메시지가 출력됩니다. 그리고 삭제 이후의 마지막 HEAD 커밋의 해시 값이 출력됩니다. 소스트리에서도 결과를 확인해 보세요.

▼ 그림 9-9 hard 옵션을 사용한 리셋 이후 상태

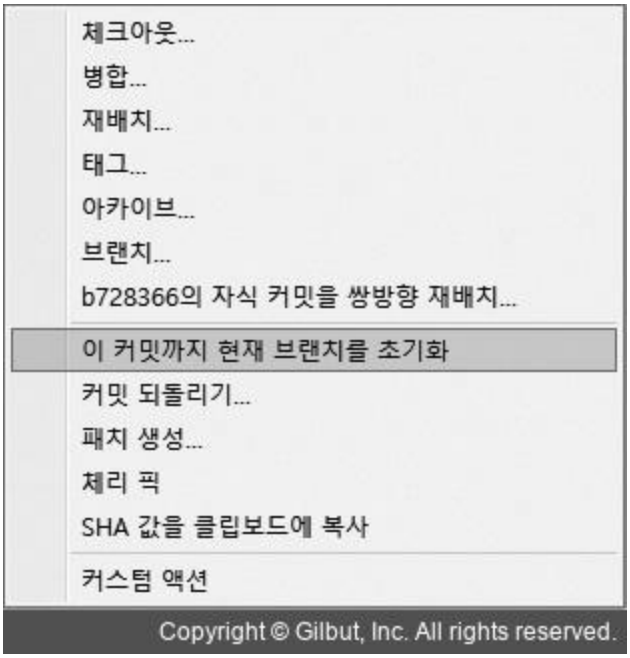


9.2.6 소스트리

리셋하려면 복귀 시점의 커밋 해시 값이 필요합니다. 하지만 매번 로그를 검색하여 리셋하는 것은 불편합니다. 소스트리를 이용하면 더 쉽게 리셋할 수 있습니다. 앞에서 이미 실습했기 때문에 여기서는 메뉴만 확인해 주세요.

먼저 소스트리의 커밋 그래프에서 복귀할 커밋을 선택합니다. 그리고 마우스 오른쪽 버튼을 눌러 이 커밋까지 현재 브랜치를 초기화 메뉴를 선택합니다.

▼ 그림 9-11 소스트리에서 초기화



9.2.7 커밋 합치기

앞에서는 리베이스 병합의 -i 옵션을 사용하면 여러 커밋을 하나로 합치는 동작을 수행할 수 있었습니다. 또 단일 커밋은 커밋 명령어의 --amend 옵션으로 커밋을 수정할 수 있었습니다. 리셋의 동작 원리를 이해하고 있다면, 커밋도 수정할 수 있습니다. 리셋의 soft 옵션은 HEAD를 해당 커밋으로 이동합니다. 그리고 원본 내용은 그대로 워킹 디렉터리에 남겨 둡니다.

실습으로 커밋을 수정해 봅시다. 수정하기 전에 먼저 로그를 확인해 보겠습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy09 (main)


$ git log --oneline 로그 확인


6619c99 (HEAD → main) menu4


b728366 menu3


f1c704f menu2


b741eef menu1


69bf712 first
```

menu3과 menu4 커밋을 reset 명령어를 사용하여 하나로 합쳐 보겠습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy09 (main)


$ git reset --soft HEAD~2 2단계 전의 커밋으로 리셋(soft 옵션)
```

최신 커밋 HEAD를 기준으로 2단계 전 상태로 리셋됩니다. 즉, menu2를 가리키는 f1c704f 커밋을 의미합니다. 로그를 다시 실행합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy09 (main)

$ git log --oneline 로그 확인

f1c704f (HEAD → main) menu2

b741eef menu1

69bf712 first
```

9.2.8 스테이지 리셋

스테이지 영역은 커밋을 하려고 임시로 결과물을 보관해 두는 공간입니다. 커밋을 하려면 워킹 디렉터리에서 작업한 내용을 스테이지 영역에 등록해야 합니다. 앞에서 배웠듯이, 스테이지 영역에 등록할 때는 add 명령어를 사용합니다.

$ git add 파일이름

또 add 명령어로 등록된 스테이지 영역의 파일을 다시 unstage 상태가 되도록 스테이징을 취소할 수 있습니다. 스테이지 영역에서 등록된 파일을 다시 unstage 상태로 만들 때는 reset 명령어를 사용합니다.

$ git reset 파일이름

reset 명령어 다음에 커밋 ID 대신 파일 이름을 사용하면 됩니다. 파일 이름을 지정하여 리셋하면 해당 파일은 unstage 상태가 됩니다. 하지만 이 명령어는 몇 가지 옵션이 축약되어 있습니다. 내부적으로 다음과 같이 여러 단계로 처리하여 파일을 unstage 상태로 변경합니다.

$ git reset --mixed HEAD 파일이름

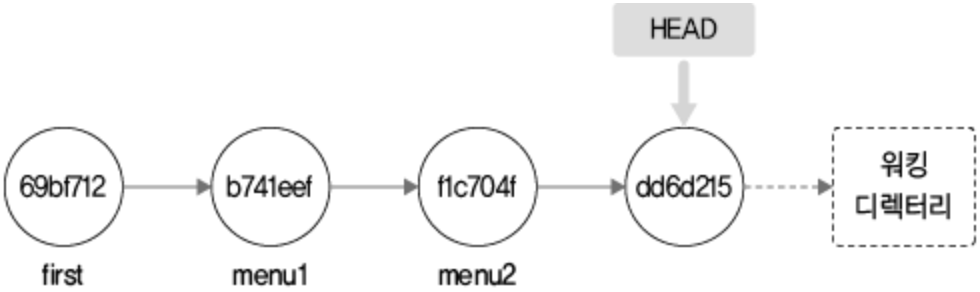
원래는 중간에 --mixed HEAD가 생략된 형태입니다. 최신 커밋에서 지정 파일을 리셋하고 mixed 옵션으로 스테이지 영역도 같이 제거합니다. 다음과 같이 HEAD 대신 다른 커밋 ID를 사용할 수도 있습니다.

$ git reset 커밋ID 파일이름

9.2.9 작업 취소

코드를 수정하는 도중 오류가 생겨 현재 작업을 모두 취소하고 싶을 수 있습니다. 보통은 다음과 같이 워킹 디렉터리에서 코드를 수정하고, 수정한 코드는 다시 스테이지 영역에 등록합니다.

▼ 그림 9-18 코드 수정과 워킹 디렉터리



수정 작업을 완전히 취소하려면 워킹 디렉터리와 스테이지 상태를 모두 제거하여 마지막 커밋 상태로 되돌려 놓아야 합니다. 그림을 보면 HEAD 포인터는 가장 마지막의 커밋 위치를 가리킵니다. 그리고 수정 작업들은 모두 워킹 디렉터리 안에 남아 있습니다. 리셋할 때의 시점을 현재 HEAD를 기준으로 하면 해당 시점의 수정 작업을 모두 삭제할 수 있습니다.

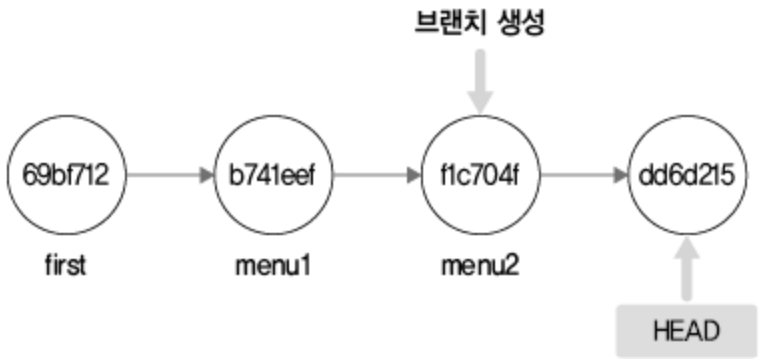
예

```
$ git reset --hard HEAD
```

9.2.10 병합 취소

리셋은 병합된 브랜치도 취소할 수 있습니다. 실습과 함께 익혀 봅시다. 먼저 실습을 위해 커밋 환경을 준비합니다. 다음과 같이 브랜치를 만들어 보겠습니다.

▼ 그림 9-19 커밋을 지정하여 브랜치 생성



menu2의 커밋 해시키를 직접 지정하여 menu 브랜치를 생성할 것입니다. 그리고 menu 브랜치로 체크아웃하겠습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy09 (main)


$ git checkout -b menu f1c704f menu2의 커밋 해시키를 직접 지정(자신의 실습에 맞는 값으로 입력)


Switched to a new branch 'menu'
```

9.2.11 주의할 점

리셋 기능을 이용하면 독립된 개인 프로젝트를 관리할 때 쉽게 이전 상태로 복귀할 수 있습니다. 하지만 저장소를 외부에 공개했거나 공유하고 있다면 주의해서 리셋을 사용해야 합니다. 작성된 커밋이 리셋으로 삭제되면 함께 작업하는 개발자에게 혼란을 줄 수 있습니다. 일반적으로 협업하거나 소스 코드를 공유한 후에는 리셋 작업을 하지 않습니다.

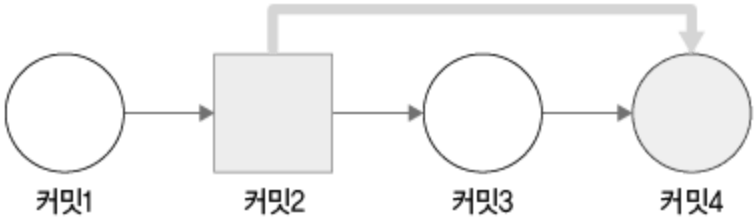
9.3 리버트

공개된 커밋은 보통 리셋 작업을 하지 않는다고 했습니다. 그러면 공개한 저장소에서는 이전 상태로 되돌리려면 어떻게 해야 할까요? 깃은 커밋의 버전을 되돌릴 수 있는 또 다른 방법인 리버트를 제공합니다. 리버트와 리셋 차이점은 커밋 정보 삭제 여부입니다.

9.3.1 취소 커밋

리셋은 기존 커밋 정보를 삭제하는 반면, 리버트는 기존 커밋을 남겨 두고 취소에 대한 새로운 커밋을 생성합니다. 취소 커밋을 생성할 때는 revert 명령어를 사용합니다. 취소 커밋은 지정한 커밋을 삭제하지 않습니다. 그 대신 삭제를 위한 새로운 커밋을 생성합니다.

▼ 그림 9-23 취소 커밋



리버트를 실습할 수 있도록 main 브랜치에서 코드를 수정한 후 커밋을 몇 개 추가하겠습니다. 먼저 menu.htm 파일에 menu5~menu7을 차례로 입력한 후 커밋합니다. code menu.htm 명령어는 생략합니다.

menu5를 추가하고 저장한 후 커밋합니다.

menu.htm

```
<ul>

    <li>menu1</li>

    <li>menu2</li>

    <li>menu3</li>

    <li>menu4</li>

    <li>menu5</li>

</ul>
```

```
infoh@DESKTOP MINGW64 /e/gitstudy09 (main)

$ git commit -am "menu5" menu5 등록 및 커밋

[main e19c0b4] menu5

 1 file changed, 1 insertion(+)
```

9.3.2 리버트 지정

직전의 커밋은 간단하게 HEAD 포인터를 이용하여 리버트했습니다. 한 번에 여러 커밋을 리버트해야 한다면 어떻게 해야 할까요? 리버트는 한 번에 커밋 하나만 취소할 수 있습니다. 따라서 여러 커밋을 리버트하려면 최신 커밋부터 순차적으로 취소해야 합니다.

그렇다면 직전의 커밋이 아닌 다른 커밋을 취소할 때는 어떻게 해야 할까요? 커밋 해시 값을 지정합니다.

$ git revert 커밋ID

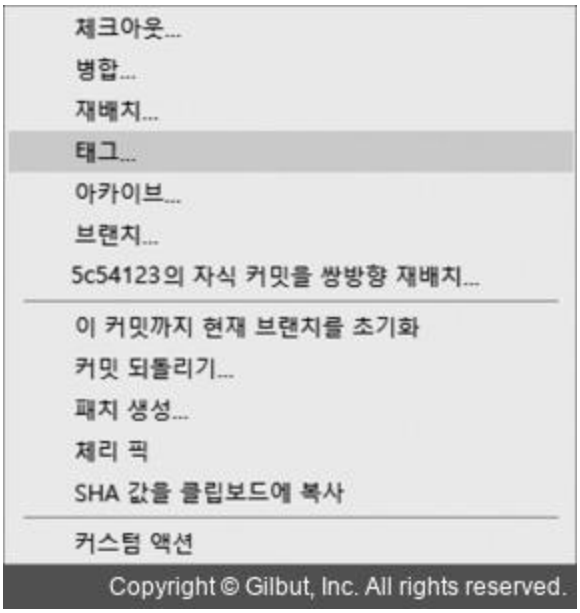
깃의 범위 지정 연산자를 사용하여 여러 커밋을 리버트할 수도 있습니다. 연산자 ..를 같이 사용합니다.

$ git revert 커밋ID .. 커밋ID

9.3.3 소스트리에서 리버트

소스트리에서 리버트하는 방법은 간단합니다. 해당 커밋을 선택한 후 마우스 오른쪽 버튼을 누릅니다. 그리고 커밋 되돌리기 메뉴를 선택합니다.

▼ 그림 9-27 커밋 되돌리기



정말 커밋을 되돌릴지 묻습니다. 되돌릴 것이라면 예를 누릅니다.

▼ 그림 9-28 리버트 확인



9.3.4 병합 취소

리버트를 이용하여 병합한 커밋을 취소할 수 있습니다. 리셋은 방금 전 실행한 병합만 삭제합니다. 하지만 리버트는 시간이 지난 후에도 과거의 병합을 선택하여 취소할 수 있습니다.

실습을 위해 menu 브랜치를 다시 병합하겠습니다. 병합 메시지 입력창이 뜨면 메시지를 입력하고 저장합니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy09 (main)


$ git merge menu 병합


Auto-merging menu.htm


Merge made by the 'recursive' strategy.


 menu.htm | 6 ++++++-


 1 file changed, 5 insertions(+), 1 deletion(-)
```

그리고 main 브랜치의 코드를 수정하고 커밋합니다. menu7을 다시 등록하겠습니다.

```
infoh@DESKTOP MINGW64 /e/gitstudy09 (main)


$ code menu.htm VS Code 실행
```

code.htm

```
<ul>

    <li>menu1

        <ul>

            <li>menu1-1</li>

        </ul>

    </li>

    <li>menu2</li>

    <li>menu3</li>

    <li>menu4</li>

    <li>menu5</li>

    <li>menu6</li>

    <li>menu7</li>

</ul>
```

```
infoh@DESKTOP MINGW64 /e/gitstudy09 (main)

$ git commit -am "menu7" 등록 및 커밋

[main da4d8e4] menu7

 1 file changed, 1 insertion(+)
```

9.3.5 리버트 히스토리

리버트를 실행하면 새 커밋이 추가되기 때문에 커밋 이력이 복잡합니다. 어떻게 보면 리셋으로 간단하게 이전 상태로 되돌리는 것이 간편해 보일 수도 있습니다. 하지만 저장소를 공개했다면 리셋으로 커밋을 삭제하는 것은 협업 차원에서 위험합니다. 이때는 리버트가 유용합니다.

9.4 정리

리셋과 리버트는 버전 관리의 특성을 그대로 보여 주는 기능입니다. 리셋과 리버트는 동작을 취소하고 과거로 돌아간다는 면에서는 유사합니다. 코드 변경 이력을 저장하고 필요한 시점에 해당 코드 상태로 쉽게 이동할 수 있습니다.

하지만 이렇게 두 명령어로 분리하여 사용하는 것은 저장소를 원격 저장소로 공유했는지 여부와 관련이 있습니다. 자신의 저장소를 외부로 공유했다면 특정 커밋을 삭제하는 것은 위험합니다. 다수의 사용자와 공유하는 저장소의 커밋 기록이 깨질 수 있기 때문입니다. 외부로 공개한 저장소라면 리버트를 사용하는 것이 좋습니다. 자신의 저장소가 컴퓨터에만 있다면 리셋을 사용해도 됩니다.

1 기본 메시지로 두고 저장한 후 빠져나와도 됩니다.